

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра ОТ

Звіт

з лабораторної роботи № 5 з дисципліни

«Бази даних»

«Нормалізація бази даних»

Виконав **ІМ-51 Марінов Владислав Віталійович**

Перевірів **Хмельницький Арсеній Андрійович**

Київ 2026

Лабораторна робота №5. Нормалізація бази даних.

Мета:

Здійснення нормалізації реляційних відношень від початкового ненормалізованого стану до 1НФ, 2НФ та 3НФ із обґрунтуванням усунення часткових і транзитивних залежностей. Розробка SQL DDL-скриптів модифікації схеми та оновлення ER-моделі.

Завдання:

1. Формалізувати та задокументувати повний набір функціональних залежностей для всіх сутностей схеми.
2. Проаналізувати вихідні таблиці на наявність аномалій вставки (Insert), модифікації (Update) та видалення (Delete).
3. Продемонструвати поетапний процес нормалізації (1НФ - 2НФ - 3НФ).
4. Розробити DDL-скрипти (команди ALTER TABLE, CREATE TABLE) для реалізації схеми в 3НФ.
5. Оновити ER-діаграму бази даних із урахуванням нових таблиць і зв'язків.
6. Перевірити коректність функціонування оновленої структури в pgAdmin.

1. Функціональні залежності (ФЗ) початкової схеми

Маємо наступні таблиці в початковій схемі: **Client, Service, Session, Additional_Service, Payment.**

1. Відношення **Client**:

- Первинний ключ: `client_id`.
- Альтернативні (потенційні) ключі: `phone_number`, `telegram_username`.
- Функціональні залежності:

`client_id` → {`telegram_username`, `full_name`, `phone_number`, `birth_date`, `medical_notes`}

`phone_number` → {`client_id`, `telegram_username`, `full_name`, `birth_date`, `medical_notes`}

2. Відношення **Service**:

- Первинний ключ: `service_id`.
- Альтернативний ключ: `name`.
- Функціональні залежності:

`service_id` → {`name`, `duration_mins`, `base_price`}

`name` → {`service_id`, `duration_mins`, `base_price`}

3. Відношення **Session**:

- Первинний ключ: `session_id`.
- Зовнішні ключі: `client_id`, `service_id`.
- Функціональні залежності:

`session_id` → {`client_id`, `service_id`, `session_date`, `start_time`, `end_time`, `status`}

4. Відношення **Additional_Service** (проблемна таблиця):

- Первинний ключ: additional_service_id.
- Зовнішній ключ: session_id.
- Функціональні залежності:

additional_service_id → {session_id, name, quantity, price, cost}

name → {price, cost}

5. Відношення **Payment** (проблемна таблиця):

- Первинний ключ: payment_id.
- Зовнішні ключі: session_id (UNIQUE), client_id.
- Функціональні залежності:

payment_id → {session_id, client_id, discount, total_amount, net_profit, payment_time}

session_id → client_id

- Транзитивна залежність: payment_id → session_id → client_id.

2. Аналіз надлишковості та аномалій у початковій схемі

Маємо аномалії у структурі двох таблиць:

Аномалії таблиці `Additional_Service`:

- **Надлишковість даних (Redundancy):** Якщо одноразовий комплект витратних матеріалів використовується у 100 різних сеансах, його назва («Одноразовий набір»), ціна продажу (0.00) та собівартість (40.00) дублюються 100 разів.
- **Аномалія вставки (Insertion Anomaly):** Неможливо зареєструвати новий витратний матеріал або додаткову послугу (наприклад, куплену преміальну олію), доки не буде створено конкретний сеанс, оскільки поле `session_id` має обмеження NOT NULL.
- **Аномалія оновлення (Update Anomaly):** У разі зміни закупівельної собівартості матеріалу майстру необхідно знайти й оновити всі історичні записи в таблиці `Additional_Service`. Помилка оновлення призведе до проблем з фінансовими даними.
- **Аномалія видалення (Deletion Anomaly):** Якщо сеанс видаляється або скасовується, разом із ним безповоротно видаляється інформація про наявність цього матеріалу в кабінеті та його вартість.

Аномалії таблиці `Payment`:

- **Транзитивна надлишковість:** Атрибут `client_id` дублюється, оскільки клієнт однозначно детермінується через `session_id` у таблиці `Session`. Зміна клієнта в сеансі вимагає синхронної зміни в оплаті, що створює ризик неузгодженості.

3. Покроковий процес нормалізації (1НФ → 2НФ → 3НФ)

Запишемо формальний перехід з ненормалізованого стану до 3НФ. Для кожного переходу задокументуємо його вимогу, попередній аналіз, потрібну дію, стан.

3.1. Перехід до Першої нормальної форми (1НФ)

- **Вимога 1НФ:** Усі атрибути повинні бути атомарними, а відношення не повинно містити повторюваних груп або масивів усередині однієї комірки.
- **Ненормалізований стан (UNF):** У паперовому журналі записів майстра додаткові матеріали та послуги записувалися в один стовпець через кому, наприклад: Одноразовий набір, Кінезіотейпування (1 шт).
- **Дія:** Кожен елемент винесено в окремий кортеж (рядок), повторювані групи ліквідовано. Поля отримали конкретні типи даних (VARCHAR, DECIMAL, INT).
- **Стан:** Відношення знаходиться в 1НФ.

3.2. Перехід до Другої нормальної форми (2НФ)

- **Вимога 2НФ:** Відношення має перебувати в 1НФ, і кожен неключовий атрибут повинен повно функціонально залежати від усього первинного ключа (відсутність часткових залежностей від частини складеного ключа).
- **Аналіз:** Якби проміжна таблиця обліку матеріалів ідентифікувалася складеним ключем (session_id, consumable_name), виникала б часткова залежність по типу:

consumable_name → {price, cost}

де параметри вартості залежать лише від назви матеріалу, а не від ідентифікатора сеансу session_id.

- **Дія:** Введення сурогатного первинного ключа additional_service_id гарантує відсутність часткових залежностей, оскільки ключ є простим (нескладеним).
- **Стан:** Відношення знаходиться в 2НФ.

3.3. Перехід до Третньої нормальної форми (3НФ)

- **Вимога 3НФ:** Відношення має перебувати в 2НФ, і жоден неключовий атрибут не повинен транзитивно залежати від первинного ключа через інший неключовий атрибут.
- **Усунення транзитивності в Additional_Service:**

Залежність: additional_service_id → name → {price, cost}.

Сутність розділяється на дві самостійні таблиці:

1. **Consumable** (довідник матеріалів і додаткових послуг):

Consumable(consumable_id, name, default_price, default_cost)

Ключ: consumable_id. Усі атрибути прямо залежать від первинного ключа.

2. **Session_Consumable** (фактичне використання матеріалів у сеансі):

Session_Consumable(session_consumable_id, session_id, consumable_id, quantity, actual_price, actual_cost)

Ключ: session_consumable_id. Поля actual_price та actual_cost фіксують ціни на момент надання послуги, усуваючи транзитивність від довідника.

- **Усунення транзитивності в Payment:**

Залежність: payment_id → session_id → client_id

З таблиці Payment видаляється стовпець client_id. Оскільки існує зв'язок one-to-one між Session та Payment, інформація про клієнта однозначно вилучається через session_id.

4. Оновлена схема даних та SQL DDL-скрипти міграції (текстом та скріншотом):

-- 1. Створення нормалізованого довідника матеріалів і послуг Consumable

```
CREATE TABLE Consumable (  
    consumable_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    name VARCHAR(100) UNIQUE NOT NULL,  
    default_price DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK  
(default_price >= 0.00),  
    default_cost DECIMAL(10,2) NOT NULL CHECK (default_cost >= 0.00)  
);
```

-- 2. Створення нормалізованої асоціативної таблиці використання розхідників

```
CREATE TABLE Session_Consumable (  
    session_consumable_id UUID PRIMARY KEY DEFAULT  
gen_random_uuid(),  
    session_id UUID NOT NULL REFERENCES Session(session_id) ON  
DELETE CASCADE,  
    consumable_id UUID NOT NULL REFERENCES  
Consumable(consumable_id) ON DELETE RESTRICT,  
    quantity INT NOT NULL DEFAULT 1 CHECK (quantity > 0),  
    actual_price DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK  
(actual_price >= 0.00),  
    actual_cost DECIMAL(10,2) NOT NULL CHECK (actual_cost >= 0.00)
```


);

-- 3. Міграція даних із минулої таблиці

```
INSERT INTO Consumable (name, default_price, default_cost)
SELECT DISTINCT name, price, cost
FROM Additional_Service;
```

-- 4. Міграція зв'язків використання розхідників

```
INSERT INTO Session_Consumable (session_id, consumable_id, quantity,
actual_price, actual_cost)
SELECT
    a.session_id,
    c.consumable_id,
    a.quantity,
    a.price,
    a.cost
FROM Additional_Service a
JOIN Consumable c ON a.name = c.name;
```

-- 5. Видалення надлишків денормалізованої таблиці

```
DROP TABLE Additional_Service CASCADE;
```

-- 6. Усунення транзитивної залежності в таблиці Payment (видалення client_id)

ALTER TABLE Payment DROP COLUMN client_id;

Скріншот коду в pgAdmin:

```
Query  Query History
1  -- 1. Створення нормалізованого довідника матеріалів і послуг Consumable
2  CREATE TABLE Consumable (
3      consumable_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4      name VARCHAR(100) UNIQUE NOT NULL,
5      default_price DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK (default_price >= 0.00),
6      default_cost DECIMAL(10,2) NOT NULL CHECK (default_cost >= 0.00)
7  );
8
9  -- 2. Створення нормалізованої асоціативної таблиці використання розхідників
10 CREATE TABLE Session_Consumable (
11     session_consumable_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
12     session_id UUID NOT NULL REFERENCES Session(session_id) ON DELETE CASCADE,
13     consumable_id UUID NOT NULL REFERENCES Consumable(consumable_id) ON DELETE RESTRICT,
14     quantity INT NOT NULL DEFAULT 1 CHECK (quantity > 0),
15     actual_price DECIMAL(10,2) NOT NULL DEFAULT 0.00 CHECK (actual_price >= 0.00),
16     actual_cost DECIMAL(10,2) NOT NULL CHECK (actual_cost >= 0.00)
17 );
18
19 -- 3. Міграція даних із минулої таблиці
20 INSERT INTO Consumable (name, default_price, default_cost)
21 SELECT DISTINCT name, price, cost
22 FROM Additional_Service;
23
24 -- 4. Міграція зв'язків використання розхідників
25 INSERT INTO Session_Consumable (session_id, consumable_id, quantity, actual_price, actual_cost)
26 SELECT
27     a.session_id,
28     c.consumable_id,
29     a.quantity,
30     a.price,
31     a.cost
32 FROM Additional_Service a
33 JOIN Consumable c ON a.name = c.name;
34
35 -- 5. Видалення надлишків денормалізованої таблиці
36 DROP TABLE Additional_Service CASCADE;
37
38 -- 6. Усунення транзитивної залежності в таблиці Payment (видалення client_id)
39 ALTER TABLE Payment DROP COLUMN client_id;
```

Також, оновлена ER-схема:

Client		
UUID	client_id	PK
VARCHAR	telegram_username	
VARCHAR	full_name	

Service		
UUID	service_id	PK

Скріншоти з pgAdmin:

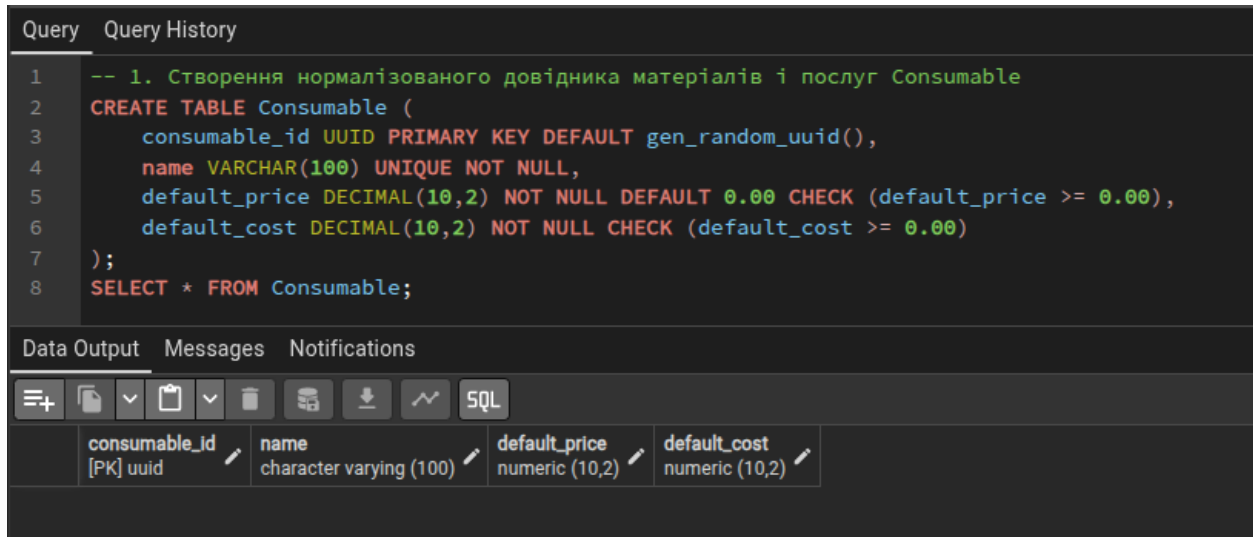


Рис. 1. Створення таблиці Consumable

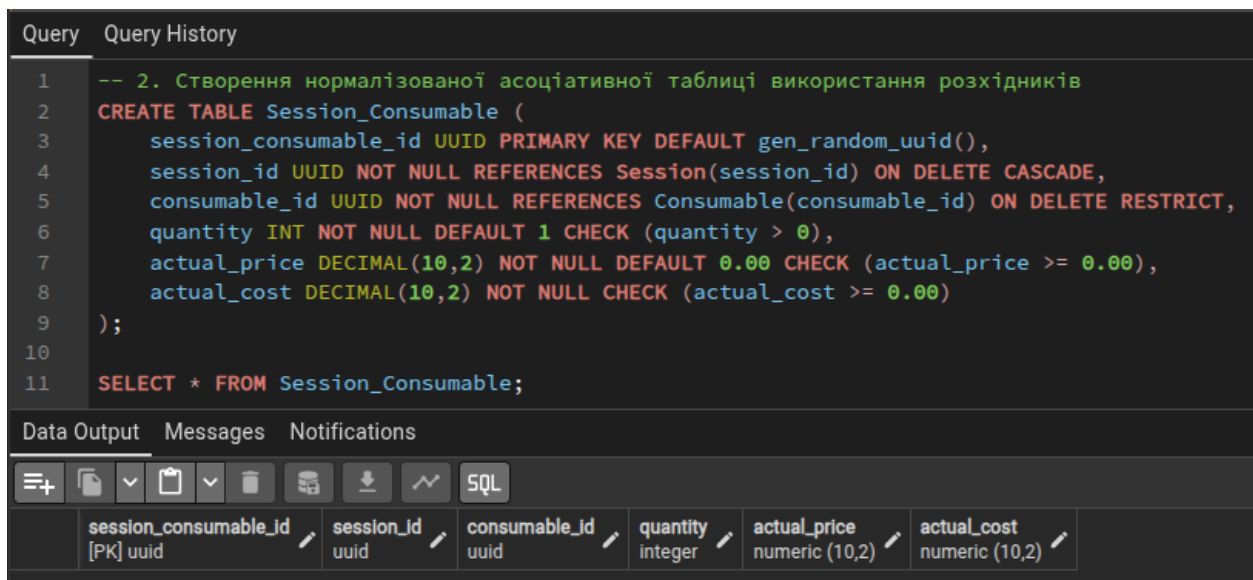


Рис. 2. Створення асоціативної таблиці Session_Consumable

Query

Query History

1

-- 3. Міграція унікальних довідникових даних із застарілої таблиці

2

INSERT INTO Consumable (name, default_price, default_cost)

3

SELECT DISTINCT name, price, cost

4

FROM Additional_Service;

5

6

SELECT * FROM Consumable;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

	consumable_id [PK] uuid	name character varying (100)	default_price numeric (10,2)	default_cost numeric (10,2)
1	71a8d29a-59a2-45d6-8a53-b4d5ae1fb...	Кінезіотейпування плеча	200.00	50.00
2	ae95dbd6-6327-4ecf-b53a-ef3a9d134b...	Преміальна олія для лімфодренажу	100.00	60.00
3	4ee26721-8fe9-471a-8add-e9beac20cd...	Одноразовий набір (простирадло, кап...	0.00	40.00

Рис. 3. Міграція довідника з усуненням дублікатів

Query

Query History

1

-- 4. Міграція історичних зв'язків використання розхідників

2

INSERT INTO Session_Consumable (session_id, consumable_id, quantity, actual_price, actual_cost)

3

SELECT

4

a.session_id,

5

c.consumable_id,

6

a.quantity,

7

a.price,

8

a.cost

9

FROM Additional_Service a

10

JOIN Consumable c ON a.name = c.name;

11

12

SELECT * FROM Session_Consumable;

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

	session_consumable_id [PK] uuid	session_id uuid	consumable_id uuid	quantity integer	actual_price numeric (10,2)	actual_cost numeric (10,2)
1	f3449587-5713-4fc9-85d6-b5fdec2bc3...	c0eebc99-9c0b-4ef8-bb6d-6bb9bd380c...	71a8d29a-59a2-45d6-8a53-b4d5ae1fb...	1	200.00	50.00
2	ecf1282f-8301-4b28-a807-c0ec102d42...	c0eebc99-9c0b-4ef8-bb6d-6bb9bd380c...	4ee26721-8fe9-471a-8add-e9beac20c...	1	0.00	40.00
3	5154fca2-7d75-4664-aea1-9729430a35...	c0eebc99-9c0b-4ef8-bb6d-6bb9bd380c...	ae95dbd6-6327-4ecf-b53a-ef3a9d134b...	1	100.00	60.00

Рис. 4. Заповнення асоціативної таблиці Session_Consumable нормалізованими даними

QueryQuery History

1

-- 5. Видалення надлишкової денормалізованої таблиці

2

DROP TABLE Additional_Service CASCADE;

3

4

SELECT table_name

5

FROM information_schema.tables

6

WHERE table_schema = 'public' AND table_type = 'BASE TABLE'

7

ORDER BY table_name;

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	table_name	🔒
	name	
1	client	
2	consumable	
3	payment	
4	service	
5	session	
6	session_consumma...	

Рис. 5. Видалення застарілої таблиці Additional_Service

QueryQuery History

1

-- 6. Усунення транзитивної залежності в таблиці Payment (видалення client_id)

2

ALTER TABLE Payment DROP COLUMN client_id;

3

4

SELECT * FROM Payment;

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	payment_id	session_id	discount	total_amount	net_profit	payment_time
	[PK] uuid	uuid	numeric (10,2)	numeric (10,2)	numeric (10,2)	timestamp without time zone
1	e0eebc99-9c0b-4ef8-bb6d-6bb9bd380e...	c0eebc99-9c0b-4ef8-bb6d-6bb9bd380c...	0.00	1000.00	910.00	2026-09-20 11:05:00
2	e0eebc99-9c0b-4ef8-bb6d-6bb9bd380e...	c0eebc99-9c0b-4ef8-bb6d-6bb9bd380c...	50.00	550.00	510.00	2026-09-20 12:50:00
3	e0eebc99-9c0b-4ef8-bb6d-6bb9bd380e...	c0eebc99-9c0b-4ef8-bb6d-6bb9bd380c...	0.00	1200.00	1140.00	2026-09-21 15:35:00

Рис. 6. Доведення таблиці Payment до 3НФ шляхом видалення client_id

Висновок:

Було здійснено аналіз для нормалізації бази даних CRM-системи. Формалізовано мінімальний набір функціональних залежностей для кожної сутності, визначено первинні та можливі ключі. Виявлено та усунено аномалії вставки, модифікації та видалення, зумовлені транзитивними залежностями в таблицях Additional_Service та Payment. За допомогою реляційної декомпозиції схему було успішно приведено до Третьої нормальної форми (3НФ). Створено незалежний довідник розхідних матеріалів Consumable, ліквідовано надлишкові вторинні зв'язки, оновлено концептуальну ER-модель та успішно верифіковано міграцію даних у PostgreSQL.